

AI 데이터 분석 · 첫걸음

파이썬 데이터 분석 문법 교과서

월드컵 예측으로 익히는 첫 도구들

함수 · 메서드 · 속성 · 키워드를 정확한 용어로

설비 고장 예측(강의) → 월드컵 승부 예측(실습)

같은 도구를, 더 친숙한 데이터로 연습합니다

2026-07-13 AI 데이터 분석 교육 정리

학습자 Minho · 국립금오공과대학교 컴퓨터공학부

이 책을 읽는 법

이 책은 오늘 만든 월드컵_예측.ipynb 노트북에 나온 코드 하나하나를, 강의 교안(01. 설비 고장을 예측하는 AI 데이터 분석)의 흐름에 맞춰 정리한 것입니다. 무작정 외우기 전에, 먼저 "이게 함수인지 메서드인지 속성인지"를 구분하는 눈부터 기릅니다. 그 구분이 0장의 핵심입니다.

분류 뱃지 — 이름 앞에 붙는 색 딱지

함수 독립 호출 **메서드** 객체.이름() **속성** 괄호 없음 **키워드** 문법 예약어

연산자 + == | **문법** f-string 등 표현식 **자료형** 리스트·딕셔너리 **셀명령** !로 시작

본문 상자 — 항목마다 이런 순서로

정의 · **형태** (문법 모양) · **예제 코드블록** · **왜 쓰나** · 그리고 필요하면 **주의** / **자주 하는 실수** 상자가 붙습니다.

개념

가장 중요한 습관 (강의 핵심): 분석은 "결과가 나왔다"에서 멈추면 안 됩니다. "**그래서 무엇을 해야 하는가**"까지 문장으로 내려야 현장에서 가치가 생깁니다. 이 책의 마지막 STEP도 "예측·판단"으로 끝나는 이유입니다.

목차

	장 / STEP	무엇을 배우나
0장	가장 먼저, 용어부터 (핵심)	함수 · 메서드 · 속성 · 키워드를 3초 만에 구분하기
지도	강의 5단계와 이 책의 STEP	수집 → 정리 → 분석 → 탐지 → 판단
STEP 0	준비 체력 — 파이썬 기본기	print · 변수 · 연산자 · 리스트 · 반복/조건 · f-string
STEP 1	수집 — 데이터 불러오기	import · read_csv · head/tail
STEP 2	정리 — 필요한 것만 남기기(필터링)	shape · 불린 인덱싱 · unique · copy · 열 추가
STEP 3	분석 — 집계하고 순위 매기기	sort_values · iloc · groupby · sum · add
STEP 4	시각화 — 그래프로 요약	figure · plot · title/label · tight_layout · show
STEP 5	판단·예측 — 나만의 도구(함수) 만들기	def · return · 딕셔너리 · round · max
부록 A	한눈에 보는 문법 사전 (치트시트)	전체 항목 요약표
부록 B	자주 하는 실수 & 코드 뜯어보기	내 노트 교정 + 노트북 버그 분석
부록 C	강의 교안과의 연결	개발환경 3가지 · 단축키 · 도구 3형제

CHAPTER 0

가장 먼저, 용어부터

"함수"라고 부르는 게 맞을까? — 분류하는 눈 기르기

노트에 적은 `df`, `copy()`, `shape()`, `head()`, `len()` ... 이걸 모두 "함수"라고 불렀는데, 사실 서로 다른 4종류가 섞여 있습니다. 이름을 정확히 알면 에러 메시지도 읽히고, 언제 괄호를 붙이는지도 저절로 정리됩니다.

0-1. 함수 vs 메서드 vs 속성 — 한 표로 끝내기

분류	무엇인가	괄호()	앞에 점(.)	이 책의 예시
함수	혼자 독립적으로 부르는 도구	있음	없음	<code>print()</code> <code>len()</code> <code>sum()</code>
메서드	어떤 값(객체)에 "붙어서" 동작	있음	있음	<code>df.head()</code> <code>df.copy()</code>
속성	객체가 "가진 정보" (동작 아님)	없음	있음	<code>df.shape</code>
키워드	파이썬 문법 예약어	없음	없음	<code>def</code> <code>for</code> <code>if</code> <code>return</code>

개념

3초 판별법 — 이름을 보면 순서대로 물어보세요.

- ① 앞에 점(.)이 있나? 없으면 → **함수** 또는 **키워드**.
 - ② 점이 있다면, 뒤에 괄호()가 붙나? 붙으면 → **메서드**, 안 붙으면 → **속성**.
- `df.shape` 은 점O·괄호X → **속성**. `df.head()` 는 점O·괄호O → **메서드**.

0-2. 나머지 용어도 짚고 가기

용어	뜻	이 책의 예시
변수 (variable)	값에 붙인 이름	<code>df</code> , <code>my_team</code> , <code>goal</code>
라이브러리 / 모듈	도구를 모아둔 꾸러미	<code>pandas</code> , <code>matplotlib</code>
연산자 (operator)	계산·비교 기호	<code>+</code> <code>*</code> <code>==</code> <code> </code>
자료형 (type)	값의 종류·담는 그릇	<code>리스트[]</code> , <code>딕셔너리{ }</code>
인자 / 매개변수	함수에 넣어주는 값 / 받는 자리	<code>team_state("Brazil")</code>

연결

강의 교안 p.71 "도구 이름 익히기"와 연결됩니다 → **pandas**(표 다루기) · **numpy**(숫자 계산) · **matplotlib**(그래프). 오늘 노트북은 이 중 pandas와 matplotlib를 씁니다.

0-3. 내 노트 교정 — 네 군데만 바로잡기

노선에 정리한 내용은 방향이 정확했습니다. 딱 네 가지만 다듬으면 완벽합니다.

내가 적은 것	올바른 표기	이유
<code>shape()</code>	<code>df.shape</code>	속성이라 괄호 없음. 괄호 붙이면 에러(호출 불가).
<code>sort_value()</code>	<code>df.sort_values()</code>	복수형 s. 여러 값을 정렬하므로 values.
<code>figure(figsize=9,4)</code>	<code>figure(figsize=(9,4))</code>	figsize는 (가로,세로) 튜플. 괄호로 묶어야 함.
<code>tight_layout</code>	<code>tight_layout()</code>	함수라 괄호 필수. 없으면 실행 안 됨.

MAP

강의 5단계와 이 책의 STEP

수집 → 정리 → 분석 → 탐지 → 판단

강의 교안 p.16은 "데이터에서 정비 판단까지"를 다섯 단계로 그림니다. 오늘 노트북의 코드도 정확히 이 흐름을 따라갑니다. 설비 데이터(온도·진동·압력) 대신 월드컵 데이터(득점)를 넣었을 뿐, 도구는 같습니다.

강의 5단계	이 책 STEP	여기서 배우는 문법
① 수집	STEP 1	<code>import</code> · <code>pd.read_csv()</code> · <code>head/tail</code>
② 정리	STEP 2	<code>shape</code> · <code>df[조건]</code> · <code>unique</code> · <code>copy</code> · 열 추가
③ 분석	STEP 3·4	<code>sort_values</code> · <code>groupby</code> · <code>sum</code> + 그래프
④ 탐지	STEP 5	<code>def</code> 로 성적 계산 도구 만들기
⑤ 판단	STEP 5	<code>predict()</code> 로 승자 예측 문장 출력

개념

STEP 0(기본기)은 이 지도 이전의 "준비운동"입니다. 교안 Part 2의 `print()`와 첫 코드 실행이 여기에 해당합니다.

STEP 0

준비 체력 — 파이썬 기본기

데이터를 만지기 전에 꼭 필요한 최소 문법

내장함수 `print()` 화면에 값을 출력

정의 괄호 안의 값을 화면에 찍어 보여주는, 파이썬에 기본 내장된 함수.

형태 `print(값1, 값2, ...)`

```
print("print를 직접 작성해봤어요!")  
print("오늘의 목표 : 월드컵 승부 예측하기")
```

- ▶ `print`를 직접 작성해봤어요!
- ▶ 오늘의 목표 : 월드컵 승부 예측하기

왜 쓰나 코드는 실행해도 결과가 저절로 보이지 않습니다. `print`로 찍어야 눈으로 확인할 수 있어요. 값을 쉼표로 나열하면 공백으로 이어 한 줄에 출력됩니다.

주의

강의 포인트: 실행했는데 "아무것도 안 나오면" 대개 정상입니다. 결과를 보고 싶으면 `print`로 직접 출력하세요. 단, 뒤에 나올 `df.head()`류는 출력 기능이 이미 들어 있어 `print`로 감쌀 필요가 없습니다.

연산자 `=` (대입) 값에 이름 붙이기

정의 오른쪽 값을 왼쪽 이름(변수)에 담는 대입 연산자. 이렇게 이름 붙인 값이 "변수"입니다.

형태 `변수이름 = 값`

```
my_team = 'South Korea'  
goal = 3  
print(my_team)    # South Korea  
print(goal)       # 3
```

왜 쓰나 값에 이름을 붙여두면 뒤에서 재사용·수정이 쉽습니다. 교안 "값에 이름 붙이기"가 바로 이것.

자주 하는 실수

`=`(대입)과 `==`(같은지 비교)는 완전히 다릅니다. `goal = 3`은 "goal에 3을 넣어라", `goal == 3`은 "goal이 3과 같니?"(참/거짓).

연산자 + - * / 산술 계산

정의 숫자를 계산하는 기호. 특히 나눗셈 `/`는 결과가 항상 소수(float)로 나옵니다.

```
print(goal + 1)    # 4
print(goal * 2)    # 6
print(goal / 4)    # 0.75
print(10 / 4)      # 2.5
```

왜 쓰나 설비된 축구든 데이터는 모두 숫자입니다. "평소와 얼마나 다른가"를 빼기·나누기로 계산하는 것이 분석의 출발점(교안 p.11·14).

주의

정수끼리 나눠도 10 / 4는 2.5처럼 소수가 됩니다. 뭘만 필요하면 //를 씁니다.

자료형 리스트 [] · 인덱싱 [i] · len() 여러 값을 순서대로

정의 리스트는 여러 값을 순서대로 담는 그릇. [i]로 i번째 값을 꺼내고(0부터 시작), len()은 개수를 세는 내장 함수입니다.

형태 목록[번호] · len(목록)

```
worldcup_teams = ['South
Korea', 'USA', 'Brazil', 'Argentina', 'France', 'England', 'Spain']
print(worldcup_teams[3])    # 배열 인덱스 지정 출력
print(len(worldcup_teams)) # 길이 출력

> Argentina
> 7
```

왜 쓰나 팀 목록·경기 목록처럼 묶음 데이터를 다룰 때 기본이 되는 그릇입니다.

자주 하는 실수

번호는 0부터 셉니다. worldcup_teams[3]은 4번째 값인 Argentina예요(South Korea=0).

키워드 for · if · == · .append() 조건에 맞는 것만 모으기

정의 for는 반복, if는 조건, ==는 같은지 비교, .append()는 리스트 끝에 값을 추가하는 메서드. 넷이 팀을 이뤄 "필터링"을 만듭니다.

```
favorite_team = []
for team in worldcup_teams:
```



```
if team == 'South Korea':
    favorite_team.append(team)
```

왜 쓰나 조건(한국인가?)에 맞는 값만 골라 새 리스트에 모으는 필터링의 원리입니다.

주의

이 4줄 흐름은 STEP 2에서 pandas의 "불린 인덱싱" 한 줄로 훨씬 짧아집니다. 원리를 먼저 이해해두면 그때 편해요.

문법 f-string `f"...{ }..."` 문장 안에 값 끼워넣기

정의 문자열 앞에 f를 붙이고 {} 안에 변수를 넣으면, 그 자리에 값이 실제로 채워집니다.

형태 `f"...{변수}..."`

```
print(f"내가 응원하는 팀은 {my_team}이고, 오늘 {goal}골을 넣었으면 좋겠어.")
```

▶ 내가 응원하는 팀은 South Korea이고, 오늘 3골을 넣었으면 좋겠어.

왜 쓰나 숫자·이름을 문장에 자연스럽게 넣어 "보고용 문장"을 만듭니다. 교안의 판단 전달("3번 설비 점검 필요")
이 바로 이 형태예요.

STEP 1

수집 — 데이터 불러오기

① 수집 · pandas가 처음 등장하는 단계

키워드 import ... as ... 도구 불러와 별명 붙이기

정의 라이브러리(도구 꾸러미)를 불러오고, as로 짧은 별명을 붙입니다.

형태 import pandas as pd

```
import pandas as pd
import matplotlib.pyplot as plt
```

왜 쓰나 매번 pandas라고 길게 쓰지 않고 pd로. 관례적으로 pandas→pd, matplotlib.pyplot→plt, numpy→np를 씁니다.

셸명령 !git clone · pip install 인터넷에서 가져오기 / 설치

정의 맨 앞의 !는 "파이썬이 아니라 터미널(셸) 명령을 실행하라"는 표시(코랩·주피터). git clone은 인터넷 저장소를 통째로 내려받습니다.

```
!git clone https://github.com/martj42/international_results.git
```

왜 쓰나 교안대로 데이터는 인터넷에서 바로 가져올 수 있습니다. pip install은 도구 자체를 설치·최신화할 때 씁니다.

함수 pd.read_csv() CSV(표) 파일 읽기

정의 CSV 파일을 읽어 표 형태의 데이터(DataFrame)로 돌려주는 pandas 함수.

형태 pd.read_csv('경로/파일.csv')

```
df = pd.read_csv('international_results/results.csv')
```

왜 쓰나 분석의 시작점. 엑셀 같은 표를 코드로 불러옵니다(교안 p.24 "불러오기"). 결과를 관례적으로 df라 부릅니다.

자료형 df (DataFrame) 표 데이터 그 자체(객체)

정의 pandas가 표를 담아두는 객체. 행(경기)과 열(항목)으로 이뤄집니다. head·shape·copy·sort_values 같은 도구들이 전부 "이 df에 붙어서" 동작합니다.

```
df.head(5)    # 잘 불러와졌는지 미리보기
```

```
▶ date   home_team  away_team  home_score  away_score  tournament ...  
▶ 1872-11-30  Scotland  England    0.0    0.0    Friendly ...
```

왜 쓰나 df가 "객체"라는 걸 알면, 왜 df.head()는 메서드이고 df.shape는 속성인지가 자연스럽게 이해됩니다. 열 = 항목(date, home_team...), 행 = 시간순 경기(교안 p.13).

메서드 df.head() · df.tail() 앞/뒤 몇 줄 미리보기

정의 표의 앞쪽(head)·뒤쪽(tail) 몇 줄만 보여주는 메서드. 숫자를 생략하면 5줄.

형태 df.head(n) df.tail(n)

```
Kor_Match[['date', 'home_team', 'away_team', 'home_score', 'away_score']].tail(10)
```

왜 쓰나 수만 줄을 전부 볼 순 없으니, 생김새와 불러오기 성공 여부를 미리보기로 확인합니다.

주의

head/tail은 출력이 내장돼 있어 print로 감싸지 않아도 표가 보입니다. 단, 셀의 "마지막 줄"일 때 자동 표시돼요.

STEP 2

정리 — 필요한 것만 남기기

② 정리 · 필터링이 핵심

속성 `df.shape` 표의 크기 (행, 열)

정의 표가 몇 행 몇 열인지 (행, 열) 형태로 알려주는 속성. 동작이 아니라 "정보"라서 괄호가 없습니다.

형태 `df.shape` → (49505, 9)

```
print("(행, 열)", df.shape) # 여러 값은 쉼표로 구분해 출력
```

```
▶ (행, 열) (49505, 9)
```

왜 쓰나 데이터가 몇 건인지(행), 항목이 몇 개인지(열)를 한 번에 파악합니다.

자주 하는 실수

`df.shape()`처럼 괄호를 붙이면 에러가 납니다(`TypeError: 'tuple' object is not callable`). `shape`는 "값"이라서 괄호가 없어요 — 노트의 `shape()`를 `shape`로 고치면 됩니다.

문법 `df[ df['열'] == 값 ]` 불린 인덱싱 = 필터링

정의 조건이 참(True)인 행만 남기는 것. STEP 0의 for+if 필터링을 pandas가 한 줄로 처리합니다.

형태 `df[df['열'] == 값]`

```
wc = df[df['tournament'] == 'FIFA World Cup']
print(f"월드컵 경기 수: {len(wc)}")
```

```
▶ 월드컵 경기 수: 1064
```

왜 쓰나 수만 건 중 월드컵 경기만 골라내기(교안 "정리"). 원리는 `df['tournament']=='FIFA World Cup'`이 행마다 True/False를 만들고, 바깥의 `df[...]`가 True인 행만 남기는 것.

연산자 | (또는) & (그리고) 여러 조건 합치기

정의 조건을 여러 개 합칠 때. |는 OR(둘 중 하나라도), &는 AND(둘 다). 각 조건은 반드시 괄호로 감쌉니다.

형태 `df[(조건A) | (조건B)]`

```
Kor_Match = wc[(wc['home_team'] == 'South Korea') | (wc['away_team'] == 'South Korea')]  
print("한국 월드컵 경기 수", len(Kor_Match))
```

▶ 한국 월드컵 경기 수 41

왜 쓰나 한국은 홈팀일 수도, 원정팀일 수도 있으니 둘 다 모아야 합니다(교안: 홈 아니면 어웨이를 합쳐 출력).

자주 하는 실수

pandas에서는 and/or가 아니라 & / |를 씁니다. 그리고 각 조건을 괄호로 꼭 감싸야 해요 — 안 그러면 우선순위 때문에 에러납니다.

메서드 `.unique()` 중복 없는 값 목록

정의 한 열에 어떤 값들이 있는지, 중복을 뺀 목록을 돌려주는 메서드.

```
korea_wc = [name for name in wc['home_team'].unique() if 'Korea' in str(name)]  
print(korea_wc)
```

▶ ['South Korea', 'North Korea']

왜 쓰나 어떤 팀·어떤 종류가 들어 있는지 파악할 때. 위 예제는 리스트 컴프리헨션과 결합해 "이름에 Korea가 든 팀"만 뽑았습니다.

문법 `[ x for ... if ... ]` 리스트 컴프리헨션

정의 for와 if를 한 줄로 압축해 새 리스트를 만드는 문법.

형태 `[표현식 for 원소 in 대상 if 조건]`

```
korea_wc = [name for name in wc['home_team'].unique() if 'Korea' in str(name)]
```

왜 쓰나 STEP 0의 for+if+append 4줄을 한 줄로. "unique 목록 중 이름에 Korea가 든 것만" 뽑아 담습니다.

메서드 `.copy()` 원본을 지키는 사본 만들기

정의 DataFrame을 복사해 별도의 사본을 만듭니다.

형태 `df.copy()`

```
wc_1st = wc.copy()  
wc_1st["total_goals"] = wc_1st["home_score"] + wc_1st["away_score"]
```

왜 쓰나 원본을 건드리지 않고 열 추가·수정 같은 실험을 사본에서 합니다. 원본 오염 방지.

주의

사본 없이 필터링 결과에 바로 열을 추가하면 pandas가 SettingWithCopyWarning 경고를 낼 수 있습니다. `copy()`가 안전한 습관.

문법 `df["새 열"] = ...` 파생 열 추가

정의 존재하지 않는 열 이름에 값을 대입하면 그 열이 새로 생깁니다.

```
wc_1st["total_goals"] = wc_1st["home_score"] + wc_1st["away_score"] # 열이 추가됨
```

왜 쓰나 총 득점 같은 "파생 지표"를 만들어 분석용 항목을 늘립니다.

자주 하는 실수

노트북 4-1의 두 번째 버전 `wc_2nd["total_goals"] = wc_2nd["home_score"] =`
`wc_2nd["away_score"]` 는 연쇄 대입이라 의도와 다르게 동작합니다. 자세한 분석은 부록 B에 있어요.

STEP 3

분석 — 집계하고 순위 매기기

③ 분석 · 요약과 정렬

메서드 `.sort_values()` 값 기준으로 정렬

정의 특정 열을 기준으로 행을 정렬하는 메서드. `ascending`으로 오름/내림을 정합니다.

형태 `df.sort_values("열", ascending=False)`

```
top = wc_1st.sort_values("total_goals", ascending=False).iloc[0]
print(top)
```

```
▶ home_team  Switzerland / away_team  Austria / total_goals  12.0  (1954 월드컵)
```

왜 쓰나 최다 득점 경기·최다 득점 국가처럼 "순위"를 낼 때. `ascending=False`면 큰 값이 위로 옵니다.

자주 하는 실수

이름은 `sort_values` — 값이 여럿이라 복수형 `s`가 붙습니다(노트의 `sort_value` 교정). `ascending`은 "오름차순으로?"의 답 → `True` 오름, `False` 내림.

속성 `.iloc[ ]` 위치(정수)로 행 선택

정의 순서 번호로 행을 고르는 인덱서. `iloc[0]`은 맨 첫 번째 행. 대괄호를 쓰지만 함수는 아닙니다.

형태 `df.iloc[정수]`

```
wc_1st.sort_values("total_goals", ascending=False).iloc[0]  # 정렬 후 1등
```

왜 쓰나 "가장 ~한 것 하나"를 뽑을 때. 정렬(`sort_values`) + `iloc[0]`은 함께 자주 쓰는 관용구입니다.

메서드 `.groupby()` 같은 값끼리 묶어 집계

정의 기준 열이 같은 행끼리 그룹으로 묶어, 그룹별로 합·평균 등을 계산합니다.

형태 `df.groupby("기준열")["대상열"].sum()`

```
home_goals = wc.groupby("home_team")["home_score"].sum()  # 팀별 홈 득점 합
away_goals = wc.groupby("away_team")["away_score"].sum()
```

왜 쓰나 팀별 총 득점처럼 "그룹 요약"을 만듭니다. 숫자 수천 개를 몇 줄로 압축(교안: 요약).

메서드 `.sum()` 합계

정의 열이나 그룹의 숫자를 모두 더합니다.

```
home["home_score"].sum() # 이 팀이 홈에서 넣은 총 골
```

왜 쓰나 총 득점·총 승수를 구할 때. 내장 `sum()`과 이름은 같지만, `df["열"].sum()`은 표 전용 메서드(대량 계산에 빠름)입니다.

메서드 `.add(fill_value=0)` 결측 채우며 더하기

정의 두 집계 결과를 더하되, 한쪽에만 있는 항목은 `fill_value`로 채워 넣고 더합니다.

형태 `a.add(b, fill_value=0)`

```
total_goals = home_goals.add(away_goals, fill_value=0)
goals_sort = total_goals.sort_values(ascending=False)
goals_sort.head(10)
```

▶ Brazil 247 · Germany 243 · Argentina 166 · France 152 · Italy 128 ...

왜 쓰나 홈 득점 합 + 원정 득점 합. 어떤 팀은 홈 기록만 있을 수 있어, `fill_value=0`으로 안전하게 합칩니다.

자주 하는 실수

그냥 `home_goals + away_goals`로 더하면, 한쪽에 없는 팀은 결과가 NaN(빈 값)이 됩니다. `fill_value=0`이 이 문제를 막아줘요.

STEP 4

시각화 — 그래프로 요약

③ 분석의 꽃 · matplotlib

강의 교안 p.15는 "완만형 이상"은 표로는 안 보이고 그래프로 흐름을 봐야 보인다고 강조합니다. 그래서 데이터를 그림으로 요약하는 능력이 함께 필요합니다.

함수 `plt.figure(figsize=(9,4))` 그림 도화지 만들기

정의 새 그래프(도화지)를 만들고 크기를 지정. `figsize=(가로, 세로)`를 인치 단위 튜플로 줍니다.

형태 `plt.figure(figsize=(가로, 세로))`

```
plt.figure(figsize=(9,4))
```

왜 쓰나 그래프 크기를 조절해 보기 좋게 만듭니다.

자주 하는 실수

`figsize`는 반드시 (9,4)처럼 괄호로 묶은 튜플. 노트의 `figsize=9,4`)는 괄호 짝과 튜플 형태가 어긋난 표기예요.

메서드 `.plot(kind='bar', color=...)` 그래프 그리기

정의 데이터를 그래프로 그리는 메서드. `kind`로 종류를 정합니다(`bar`=막대).

```
top10 = goals_sort.head(10)
top10.plot(kind='bar', color="teal")
```

왜 쓰나 상위 10개국 득점을 막대로 비교. 숫자 표를 그림 하나로 요약합니다(교안).

함수 `plt.title()` · `xlabel()` · `ylabel()` 제목·축 이름

정의 그래프의 제목과 x축·y축 이름을 붙이는 함수들.

```
plt.title("Top 10 Goal-Scoring Nation in World Cup")
plt.xlabel("Nation")
plt.ylabel("Goals")
```

왜 쓰나 그래프만으론 무엇을 뜻하는지 알 수 없습니다. 라벨이 있어야 "보고용" 그림이 됩니다(강의: 타이틀 = 제목).

함수 plt.tight_layout() 여백 자동 정리

정의 제목·라벨이 잘리지 않도록 여백을 자동으로 정리하는 함수.

```
plt.tight_layout() # 여백 자동 정리
```

왜 쓰나 축 이름이 그림 밖으로 잘려나가는 것을 막아줍니다.

자주 하는 실수

함수라서 괄호가 꼭 필요합니다. plt.tight_layout처럼 괄호를 빼면 "함수 자체"만 가리킬 뿐 실행되지 않아요.

함수 plt.show() 그래프 표시

정의 지금까지 그린 그래프를 실제로 화면에 띄우는 함수.

```
plt.show() # 보여주기
```

왜 쓰나 print가 값을 보여주듯, show()를 불러야 완성된 그림이 화면에 나타납니다.

STEP 5

판단·예측 — 나만의 도구 만들기

④ 탐지 · ⑤ 판단 · def로 함수 제작

키워드 def 나만의 함수 정의

정의 내가 직접 쓸 함수(도구)를 만드는 키워드. def 아래 들여쓴 블록이 함수의 몸통입니다.

형태 def 함수이름(입력) :

```
def team_state(team):  
    home = df[df["home_team"] == team]  
    away = df[df["away_team"] == team]  
    ...
```

왜 쓰나 같은 작업(팀 성적 계산)을 팀 이름만 바꿔 재사용합니다(교안: 도구 만들기).

주의

괄호 안 team은 매개변수(parameter, 받는 자리). 부를 때 넣는 team_state("Brazil")의 "Brazil"은 인자(argument, 넣는 값)입니다.

키워드 return 결과를 돌려주기

정의 함수가 계산한 결과를 밖으로 돌려주는 키워드.

```
return {  
    "games": int(games),  
    "win_rate": round(float(wins / games), 3),  
    "avg_goals": round(float(goal_for / games), 2),  
}  
  
▶ team_state('Brazil') → {'games':1064, 'win_rate':0.634, 'avg_goals':2.18}
```

왜 쓰나 함수 밖에서 그 결과를 받아 다시 쓰려면 return이 필요합니다.

자주 하는 실수

print vs return: print는 화면에 보여주고 사라집니다. return은 값을 "넘겨"주어 다른 계산에 재사용됩니다.
predict()가 team_state()의 return 값을 받아 쓰는 게 그 예.

자료형 딕셔너리 { "키": 값 } 이름표로 값 묶기

정의 이름표(키)로 값을 묶어 담는 그릇. dict["키"]로 꺼냅니다.

```
{ "games": 1064, "win_rate": 0.634, "avg_goals": 2.18 }  
# 꺼낼 때: a["win_rate"] → 0.634
```

왜 쓰나 여러 결과(경기수·승률·평균득점)를 한 묶음으로 반환할 때 깔끔합니다.

내장함수 round() · max() · int() · float() 반올림·최댓값·형변환

정의 round(반올림), max(가장 큰 값), int/float(정수·실수로 변환) — 모두 파이썬 내장 함수.

형태 round(값, 자릿수) · max(a, b)

```
prob_a = a["win_rate"] / (a["win_rate"] + b["win_rate"])  
edge = round(max(prob_a, prob_b) * 100) # 우세한 쪽 확률만
```

왜 쓰나 round로 승률·평균을 보기 좋게, max로 두 팀 확률 중 큰 쪽(우세팀 확률)만 뽑습니다(노트북 주식: 둘 중 더 큰 값 반환).

내장함수 sum() vs + 노트에 적어둔 그 구분, 정리

정의 노트에 "+는 앞뒤를 더하고, sum은 여러 개를 다 합"이라고 적었는데 정확합니다. 조금만 더 다듬으면:

```
a + b # 값 두 개를 더함  
sum([1, 2, 3]) # 리스트 안 여러 값을 한 번에 합 → 6
```

왜 쓰나 +는 두 값, sum()은 "반복 가능한 묶음(리스트 등)의 원소를 모두" 합산합니다. 참고로 df["열"].sum()은 같은 개념의 표 전용 메서드예요.

STEP 5 마무리 — predict() 전체 흐름

두 도구가 이어집니다. team_state()가 팀의 (경기수·승률·평균득점)을 계산해 딕셔너리로 돌려주면, predict()가 두 팀의 승률을 비교해 이길 확률과 승자를 문장으로 출력합니다.

```
def predict(team_a, team_b):  
    a = team_state(team_a)  
    b = team_state(team_b)  
    prob_a = a["win_rate"] / (a["win_rate"] + b["win_rate"])  
    prob_b = b["win_rate"] / (a["win_rate"] + b["win_rate"])  
    winner = team_a if prob_a > 0.5 else team_b  
    edge = round(max(prob_a, prob_b) * 100)  
    print(f'-> 예측 승자: {winner} 약 {edge}% 우세')
```

▶ `predict('Argentina', 'Switzerland')` → 예측 승자: Argentina 약 61% 우세

개념

이 예측은 "승률 비율"만 쓰는 아주 단순한 규칙입니다. 무승부·상대 전적·최근 폼은 고려하지 않아요. 그래도 강의가 말한 흐름 — 데이터 → 계산 → 판단 문장 — 을 완성했다는 점이 핵심입니다.

한눈에 보는 문법 사전

전체 항목 요약 — 시험·복습용 치트시트

분류	이름	형태	한 줄 설명
내장함수	print()	print(a, b)	화면에 값 출력
연산자	=	이름 = 값	값에 이름 붙이기(대입)
연산자	== &	a == b	비교 / OR / AND
자료형	리스트[]	목록[i]	순서대로 담고 i번째 꺼내기
내장함수	len()	len(x)	개수 세기
키워드	for / if	for x in ...	반복 / 조건
문법	f-string	f"{x}"	문장에 값 끼우기
키워드	import as	import pandas as pd	도구 불러와 별명
함수	pd.read_csv()	read_csv('f.csv')	CSV를 표로 읽기
메서드	.head()/.tail()	df.head(n)	앞/뒤 미리보기
속성	.shape	df.shape	(행, 열) 크기
문법	불린 인덱싱	df[df['c']==v]	조건 맞는 행만(필터)
메서드	.unique()	df['c'].unique()	중복 없는 값 목록
메서드	.copy()	df.copy()	사본 만들기
메서드	.sort_values()	sort_values("c")	값 기준 정렬
속성	.iloc[]	df.iloc[0]	위치로 행 선택
메서드	.groupby()	groupby("c")	묶어서 집계
메서드	.sum()	df["c"].sum()	합계
메서드	.add()	a.add(b, fill_value=0)	결측 채우며 더하기
함수	plt.figure()	figure(figsize=(9,4))	그림 도화지
메서드	.plot()	plot(kind='bar')	그래프 그리기
함수	plt.title()	title(), xlabel()	제목·축 이름
함수	plt.tight_layout()	tight_layout()	여백 정리

분류	이름	형태	한 줄 설명
함수	<code>plt.show()</code>	<code>show()</code>	그래프 표시
키워드	<code>def / return</code>	<code>def f(x):</code>	함수 정의 / 결과 반환
자료형	딕셔너리{ }	<code>{"k": v}</code>	이름표로 값 묶기
내장함수	<code>round()/max()</code>	<code>round(x, 3)</code>	반올림 / 최댓값

부록 B

자주 하는 실수 & 코드 뜯어보기

내 노트 교정 + 노트북 속 버그 분석

B-1. 표기 실수 4종 (0-3장 요약)

✗ 틀린 표기	✓ 올바른 표기	한 줄 이유
<code>df.shape()</code>	<code>df.shape</code>	속성 → 괄호 없음
<code>sort_value()</code>	<code>sort_values()</code>	복수형 s
<code>figsize=9,4)</code>	<code>figsize=(9,4)</code>	(가로,세로) 튜플
<code>tight_layout</code>	<code>tight_layout()</code>	함수 → 괄호 필수

B-2. 노트북 4-1의 연쇄 대입 버그

두 번째 버전(wc_2nd)은 총 득점을 잘못 계산합니다. 왜 그런지 한 줄씩 뜯어봅시다.

```
# ✗ 노트북에 적힌 코드
wc_2nd["total_goals"] = wc_2nd["home_score"] = wc_2nd["away_score"]

# 파이썬은 오른쪽 값을 왼쪽 둘 "모두"에 넣습니다:
#   total_goals ← away_score,   home_score ← away_score
#   즉 total_goals = 원정 득점 뿐! (홈+원정 아님)   게다가 home_score도 덮어써짐

# ✓ 올바른 코드 (4-1 첫 버전처럼)
wc_2nd["total_goals"] = wc_2nd["home_score"] + wc_2nd["away_score"]
```

개념

출력이 8.0으로 "맞아 보였던" 이유는 그 경기(쿠바 8 : 8 스웨덴)가 마침 동점이라 home=away였기 때문입니다. 값이 다른 경기에선 곧바로 틀린 결과가 나옵니다. "우연히 맞는" 코드가 가장 위험해요.

B-3. f-string 안에서 따옴표 겹칠 때

```
# ✗ 바깥도 " , 안쪽도 " → 파이썬 3.12 미만에서 에러
```



```
print(f"{team_a}: 승률: {a['win_rate']}")
```

```
# ✓ 안전하게: 안쪽은 작은따옴표로
```

```
print(f"{team_a}: 승률: {a['win_rate']}")
```

주의

노트북은 최신 파이썬(3.12+)이라 통과했지만, 회사·서버의 예전 버전에선 에러가 날 수 있습니다. 따옴표는 바깥/안쪽을 다르게 쓰는 습관이 안전합니다.

부록 C

강의 교안과의 연결

노트북 밖, 교안에서 함께 익힌 것들

C-1. 개발 환경 3가지

환경	설치	실행 위치	특징
Colab	불필요	구글 컴퓨터	브라우저만 열면 시작. 입문·테스트용
Jupyter	필요	내 컴퓨터	셀 단위 실행. 코랩과 사용법 유사
VSCode	필요	내 컴퓨터	현장에서 가장 많이 쓰는 전문 도구

연결

오늘은 가장 쉬운 **Colab**으로 시작했고, 노트북 파일은 `.ipynb` (셀 단위 실행) 형식입니다. 내일부터 쓸 `.py`는 파일 전체가 위→아래로 한 번에 실행돼요.

C-2. 꼭 외우는 실행 단축키 (코랩·주피터)

단축키	기능
Shift + Enter	셀 실행 후 다음 셀로 (가장 많이 씀)
Ctrl + Enter	셀 실행하고 그 자리에 머무르기
Ctrl + M, A / B	위(Above) / 아래(Below)에 새 셀 추가
Ctrl + M, D	선택한 셀 삭제

C-3. 데이터 3형제 & 습관

도구	역할
pandas	표 데이터를 다루는 도구 (오늘 주력)
numpy	대량의 숫자 계산을 빠르게
matplotlib	숫자를 그래프로 요약 (STEP 4)

개념

강의가 가장 강조한 습관: 결과가 나온 뒤 **"그래서 무엇을 점검·행동해야 하나"**까지 문장으로 내려야 합니다.
print로 결과를 꼭 확인하고, 예측·판단으로 마무리하세요.

맺음말 — 다음 한 걸음

오늘 익힌 pandas 필터링·집계·시각화와 def로 함수 만들기는, 데이터 분석뿐 아니라 백엔드(FastAPI로 데이터 API 만들기)로도 그대로 이어집니다. 정보처리기사와 "프로그래밍 언어 활용"·"데이터베이스" 과목과도 맞닿아 있으니, 이 문법들을 손에 익혀두면 두 마리 토끼를 잡을 수 있어요.

개념

복습 루틴 추천: ①부록 A 치트시트로 분류(함수/메서드/속성) 먼저 떠올리기 → ②노트북을 위에서 아래로 다시 실행하며 각 줄이 어느 STEP인지 말해보기 → ③ predict()의 두 팀을 바꿔가며 결과 해석하기. "이 결과로 무엇을 말할 수 있나"까지 한 문장씩 붙이면 강의가 말한 실무 감각이 붙습니다.

— 끝 —